Ejercicio 1. Método signo

1. Análisis del método signo

- **¿Qué hace?**: Básicamente, nos dice si un número es positivo, negativo o si es cero.
- **Entradas**: Recibe un número entero (x).
- **Salidas**: Nos devuelve un "código": 1 (positivo), -1 (negativo) o 0 (cero).
- **Casos normales y límite**: Los normales serían cualquier número como 7 o -3. El caso límite clave aquí es el **0**, porque es donde el programa decide cambiar de un camino a otro.

2. Complejidad ciclomática

contar cuántas "decisiones" toma el código. Imagina que el código es una carretera con desvíos:

1. **Primer desvío**: ¿Es el número mayor que 0? (if (x > 0))
2. **Segundo desvío**: ¿Es el número menor que 0? (else if (x < 0))
3. **Camino final**: Si no ha ido por ninguno de los anteriores, solo le queda llegar al final (return 0).

Cálculo: Tenemos 2 decisiones. La regla dice que la complejidad es el número de decisiones + 1.

2+1=3

O sea, que como mínimo necesito **3 pruebas** para haber pasado por todos los rincones del código.

3. Diseño de casos de prueba

ID	DESCRIPCIÓN	ENTRADA	ESPERADO	ESTADO
TC01	Número positivo	x = 10	1	if (x > 0)
TC02	Número negativo	x = -5	-1	else if (x < 0)
TC03	Valor cero	x = 0	0	final return

Ejercicio 2. Método clasificarEdad

1. Análisis del método clasificarEdad

- **¿Qué hace?**: Clasifica a una persona en una etapa de la vida según los años que tenga. Además, controla que no le metan números negativos (porque no existen edades negativas).
- **Entradas**: Un número entero (edad).
- **Salidas**: Una palabra (String) con la categoría ("Infancia", "Niñez", etc.) o lanza un error si la edad es menor que 0.
- **Casos normales y límite**:
  - **Normales**: Cualquier edad típica (ej. 20 años).
  - **Límites**: Aquí hay muchos porque cada vez que saltamos de categoría (a los 6, 12, 18, 25 y 60 años) hay un límite. También el **0** (mínimo) y el **-1** (primer error).

2. Complejidad Ciclomática (Explicación natural)

Para calcularla sin líos, contamos cuántas veces el código tiene que elegir un camino:

1. **Decisión 1**: ¿Es menor que 0? (if (edad < 0)) -> Salta el error.
2. **Decisión 2**: ¿Es menor que 6? (if (edad < 6))
3. **Decisión 3**: ¿Es menor que 12? (else if (edad < 12))
4. **Decisión 4**: ¿Es menor que 18? (else if (edad < 18))
5. **Decisión 5**: ¿Es menor que 25? (else if (edad < 25))
6. **Decisión 6**: ¿Es menor que 60? (else if (edad < 60))
7. **Camino final**: Si no es ninguna de las anteriores, es "Vejez".

Cálculo: Tenemos 6 puntos de decisión. Aplicamos la regla de P+1:

6+1=7

Necesito **7 pruebas** para estar seguro de que he pasado por todos los if y el else final.

3. Diseño de casos de prueba

ID	DESCRIPCIÓN	ENTRADA	ESPERADO	ESTADO
TC01	Edad negativa	-1	egalArgumentExcepti	
TC02	ímite inferior infancia	0	"Infancia"	
TC03	Límite superior niñez	11	"Niñez"	
TC04	ransición adolescenci	17	"Adolescencia"	
TC05	Transición juventud	24	"Juventud"	
TC06	Adultez plena	40	"Adultez"	
TC07	Límite vejez	60	"Vejez"	
TC08	Caso límite superior	120	"Vejez"	

Ejercicio 3

1. Análisis del método contarPositivos

- **¿Qué hace?**: Recorre una lista de números (un array) y cuenta cuántos de ellos son mayores que cero.
- **Entradas**: Un array de enteros (int[] datos).
- **Salidas**: Un número entero que representa el total de positivos encontrados.
- **Casos normales y límite**:
  - **Normales**: Un array con números variados.
  - **Límites**: Un array vacío (longitud 0), un array donde ningún número sea positivo, o un array que incluya el **0** (que no debe contarse como positivo).

2. Complejidad ciclomática

1. **El bucle for**: Es una decisión. En cada vuelta, el programa pregunta: "¿He llegado ya al final del array?". Si la respuesta es sí, sale del bucle; si es no pues se sigue.
2. **El if dentro del bucle**: Es la segunda decisión. Pregunta: "¿Es este número concreto mayor que cero?".

Cálculo: Tenemos 2 puntos de decisión (el for y el if). Aplicamos la fórmula P+1:

2+1=3

Necesitamos **3 pruebas** para cubrir los caminos principales.

3. Diseño de casos de prueba

ID	DESCRIPCIÓN	ENTRADA	ESPERADO	ESTADO
TC01	Array vacío	int[] {}	0	
TC02	Todo negativos	int[] {-2, -5}	0	
TC03	Mezcla (Pos, Neg, 0)	int[] {1, -1, 0}	1	

Ejercicio 4

1. Análisis del método

- **¿Qué hace?**: Transforma una nota numérica (0-10) en su equivalente en texto (Suspenso, Bien, etc.).
- **Entradas**: Un número entero (nota).
- **Salidas**: Una cadena de texto (String) o una excepción si la nota se sale del rango 0-10.
- **Casos normales y límite**:
    - **Normales**: Notas típicas como 5 o 8.

- **Límites**: El **-1** y el **11** (para probar que salta el error), y los valores donde cambia la calificación (el **4** y el **5**, el **6**, el **8** y el **9**).

2. Complejidad ciclomática

Aquí la complejidad viene de cuántas condiciones tiene que mirar el if gigante que hemos montado. Imagínate que eres el ordenador y vas leyendo de izquierda a derecha:

1. **Primera parada**: ¿Se puede dividir por 4? (anyo % 4 == 0).
2. **Segunda parada**: ¿**NO** se puede dividir por 100? (anyo % 100 != 0).
3. **Tercera parada**: ¿O quizás se puede dividir por 400? (anyo % 400 == 0).

La cuenta: Tenemos 3 "preguntas" (condiciones). Como siempre, sumamos 1 al número de preguntas:

3+1=4

3. Diseño de casos de prueba

ID	DESCRIPCIÓN	ENTRADA	ESPERADO	ESTADO
TC01	Límite inferior inválido	-1	egalArgumentExcepti	Validación (< 0)
TC02	ímite superior inválid	11	egalArgumentExcepti	Validación (<10)
TC03	Rango suspenso	3	SUSPENSO	case 0, 1, 2, 3, 4
TC04	Valor suficiente	5	"SUFICIENTE"	case 5
TC05	Valor bien	6	"BIEN"	case 6
TC06	Rango notable	8	"NOTABLE"	case 7, 8
TC07	Rango sobresaliente	10	"SOBRESALIENTE"	default

Ejercicio 5

1. Análisis del método

- El código decide si un año es bisiesto. Parece fácil (divisible por 4), pero tiene "trampa" con los años que acaban en 00.
- **Entradas**: Un número entero (anyo).
- **Salidas**: Un sí o un no (true o false).
- **Casos para no pillarse los dedos**:
- **El normal**: 2024 (bisiesto de toda la vida).
- **El** 2100 (parece bisiesto porque es divisible por 4, pero como es centenario y no divisible por 400, no lo es).
- **El** 2000 (es centenario, pero como también se puede dividir por 400, este sí es bisiesto).

2. Complejidad Ciclomática

Aquí la complejidad viene de cuántas condiciones tiene que mirar el if gigante que hemos montado. Imagínate que eres el ordenador y vas leyendo de izquierda a derecha:

1. **Primera parada**: ¿Se puede dividir por 4? (anyo % 4 == 0).
2. **Segunda parada**: ¿**NO** se puede dividir por 100? (anyo % 100 != 0).
3. **Tercera parada**: ¿O quizás se puede dividir por 400? (anyo % 400 == 0).

La cuenta: Tenemos 3 "preguntas" (condiciones). Como siempre, sumamos 1 al número de preguntas:

3+1=4

Así que con **4 pruebas** sirven

3. Diseño de casos de prueba

ID	DESCRIPCIÓN	ENTRADA	ESPERADO	ESTADO
TC01	visible por 4, no por 1	2024	true	Cubre: `(T && T)
TC02	visible por 4, no por 1	2100	false	Cubre: `(T && F)
TC03	Divisible por 400	2000	true	Cubre: `(T && F)
TC04	No divisible por 4	2023	false	Cubre: `(F && X)

Ejercicio 6

1. Análisis del método

- **¿Qué hace?**: Pues el juego de toda la vida. Mira lo que han sacado los dos jugadores, comprueba que sea válido y dice quién ha ganado o si han empatado.
- **Entradas**: Dos textos (jugador1 y jugador2). Lo bueno es que el código usa .toUpperCase(), así que da igual si escribes "piedra" o "PIEDRA".
- **Salidas**: Un mensaje diciendo "GANA J1", "GANA J2", "EMPATE" o un error si te inventas la jugada.
- **Casos para ir con ojo**:
- **El nulo**: Si alguien pasa un null por error.
- **El tramposo**: Alguien que escribe "LAGARTO".
- **El empate**: Los dos sacan lo mismo.
- **Las 3 victorias de J1**: Piedra-Tijera, Tijera-Papel y Papel-Piedra.
- **Las 3 victorias de J2**: (Que es básicamente cuando no ocurre nada de lo anterior).

2. Complejidad Ciclomática

contar las "decisiones" que toma el ordenador:

1. **¿Hay algún nulo?**: (jugador1 == null || jugador2 == null). Aquí hay **2** preguntas (una por cada jugador).
2. **¿La jugada es válida?**: (!j1.matches(...) || !j2.matches(...)). Otras **2** preguntas.
3. **¿Es un empate?**: (j1.equals(j2)). **1** pregunta.
4. **¿Gana el Jugador 1?**: Aquí hay **3** combinaciones posibles (PIEDRA-TIJERA, TIJERA-PAPEL, PAPEL-PIEDRA).

La cuenta: Tenemos 2+2+1+3=8 puntos de decisión. Aplicamos la regla de P+1:

8+1=9

Necesitamos **9 pruebas** para estar 100% seguros de que hemos pasado por cada rincón del código y que todo funciona.

3. Diseño de casos de prueba

ID	DESCRIPCIÓN	ENTRADA 1	ENTRADA 2	ESPERADO	RAMA
TC01	Jugador 1 es nulo	null	PIEDRA	Exception	Validación de nulos
TC02	Jugada inventada	PIEDRA	FUEGO	Exception	Validación de palabras
TC03	Los dos igual	PAPEL	PAPEL	EMPATE	Lógica de empate
TC04	Piedra rompe tijera	PIEDRA	TIJERA	GANA J1	Victoria J1 (opción 1)
TC05	Tijera corta papel	PAPEL	PAPEL	GANA J1	Victoria J1 (opción 2)
TC06	Papel envuelve piedra	TIJERA	PIEDRA	GANA J1	Victoria J1 (opción 3)
TC07	J2 gana con Piedra	PAPEL	PIEDRA	GANA J2	Salida final (J2 gana)
TC08	J2 gana con Papel	PIEDRA	PAPEL	GANA J2	Salida final (J2 gana)
TC09	J2 gana con Tijera	PAPEL	TIJERA	GANA J2	Salida final (J2 gana)

.